

Data Imbalance in Machine Learning

BY

Dr. Adnan Amin

Assistant Professor / Program Director (CS)

School of Computer Science and IT

IMSciences, Peshawar.

Undersampling Methods For Imbalanced Data

Learning Objectives

- After completing this lecture, students will be able to:
 - Understand the problem of class imbalance
 - Explain why imbalance harms machine learning models
 - Define undersampling
 - Differentiate undersampling from oversampling
 - Explain advantages and disadvantages of undersampling
 - Identify real-world applications of undersampling

What is Undersampling?

- Instead of adding data (like oversampling), we remove some majority class data (i.e., undersampling).
- Undersampling (also called downsampling) is a technique used to handle imbalanced datasets by reducing the number of samples in the majority class. OR
- Undersampling is a technique that reduces the number of majority class samples in order to balance class distribution.

Why Imbalance is Dangerous

- Models become biased toward majority class
- Minority class is ignored
- High accuracy becomes misleading
- Poor recall for minority class
- Failure in critical applications
- Example:
 - 99% accuracy but zero fraud detected

Advantages of Undersampling

- Faster Training
- Reduced Storage Cost
- Lower Computational Complexity
- Reduced Overfitting Risk
- Better Focus on Minority Class
- Useful for Massive Datasets
- Industry Importance:
 - Cloud cost reduction
 - Faster model deployment

Disadvantages of Undersampling

- Information Loss
- Risk of Underfitting
- Important Samples May Be Removed
- Decision Boundary May Change
- Reduced Generalization

When to Avoid Undersampling

- Very small datasets
- Majority class is important
- Severe information loss
- Overfitting or underfitting risks
- High false positive cost

General Workflow of Undersampling

- **Pipeline**

- Load Dataset
- Analyze Class Distribution
- Select Undersampling Technique
- Resample Dataset
- Train Model
- Evaluate Performance

Types of Undersampling

- **Two Major Categories**

- 1. Fixed Method**

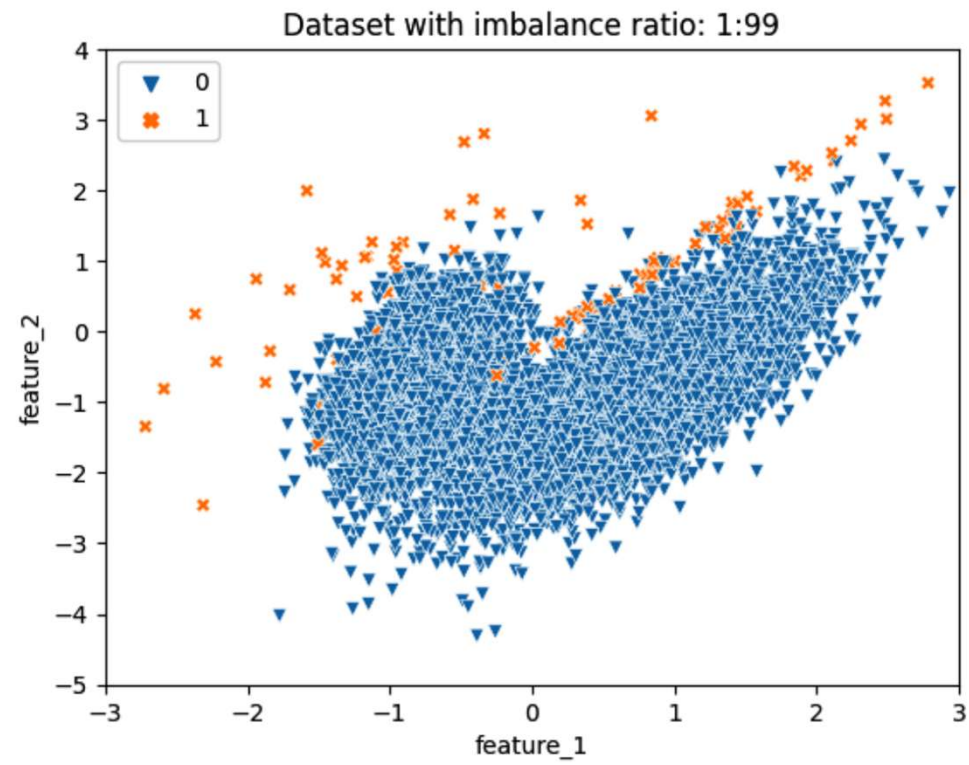
- we reduce the number of majority class samples to the size of the minority class.

- 2. Cleaning Method**

- the number of samples of the majority class is reduced based on some pre-determined criteria, independent of the absolute number of examples
 - Removes noisy or redundant samples

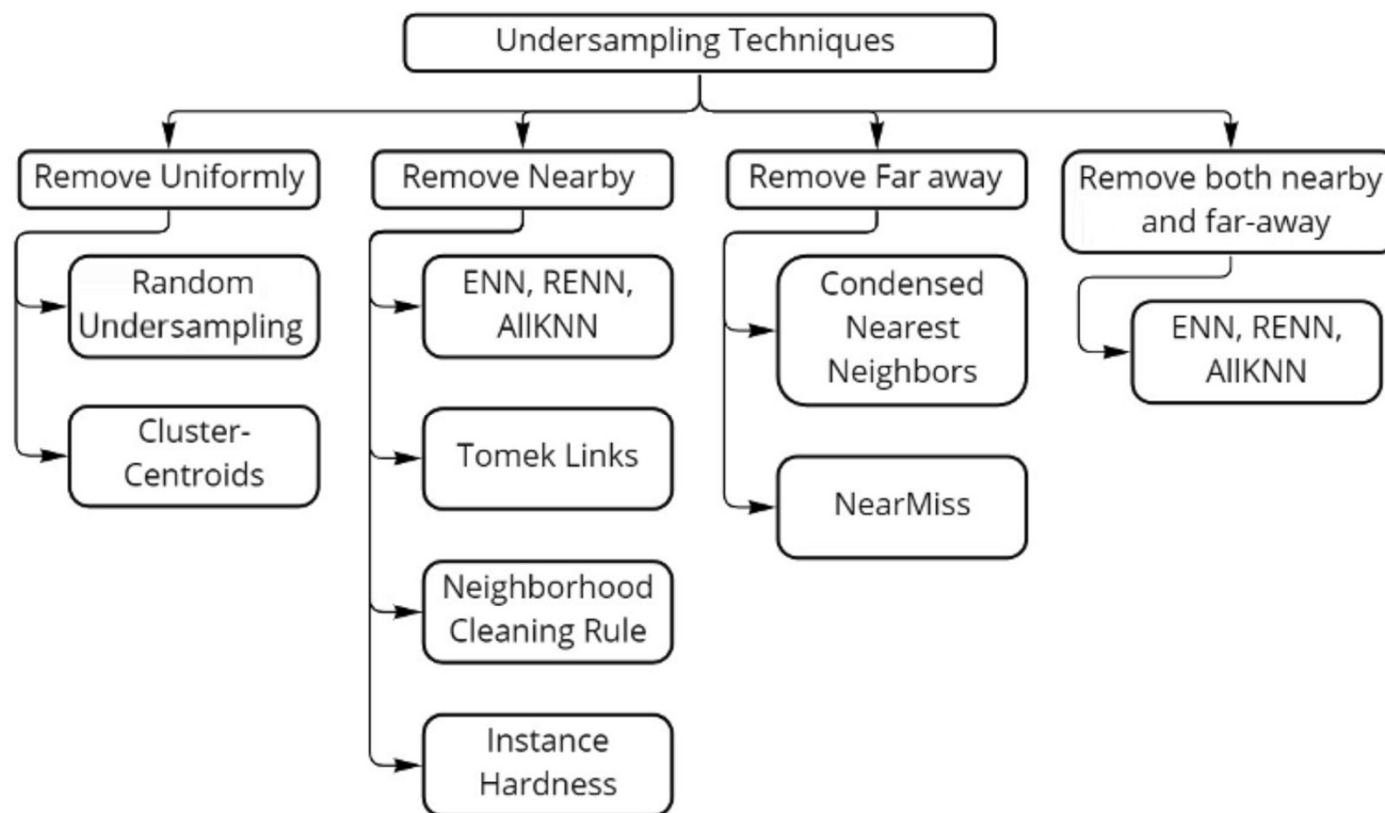
- **Examples:**

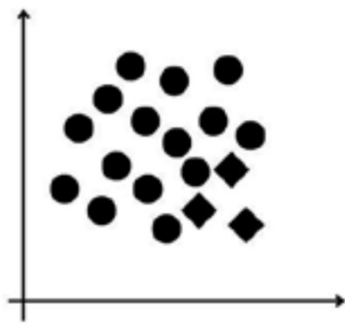
- RandomUnderSampler
 - Tomek Links
 - ENN
 - NCR



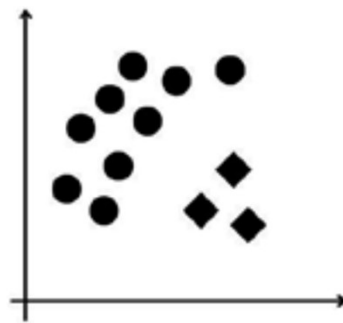
- Plotting a dataset with an imbalance ratio of 1:99

Three Undersampling Strategies

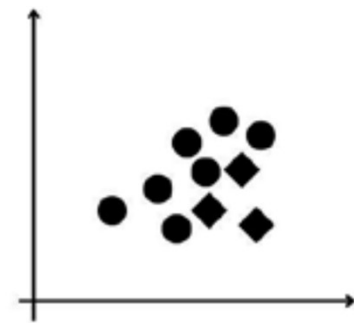




a



b



c

a) Imbalanced data with two classes, b) After removing noisy samples, c) After removing samples away from the boundary

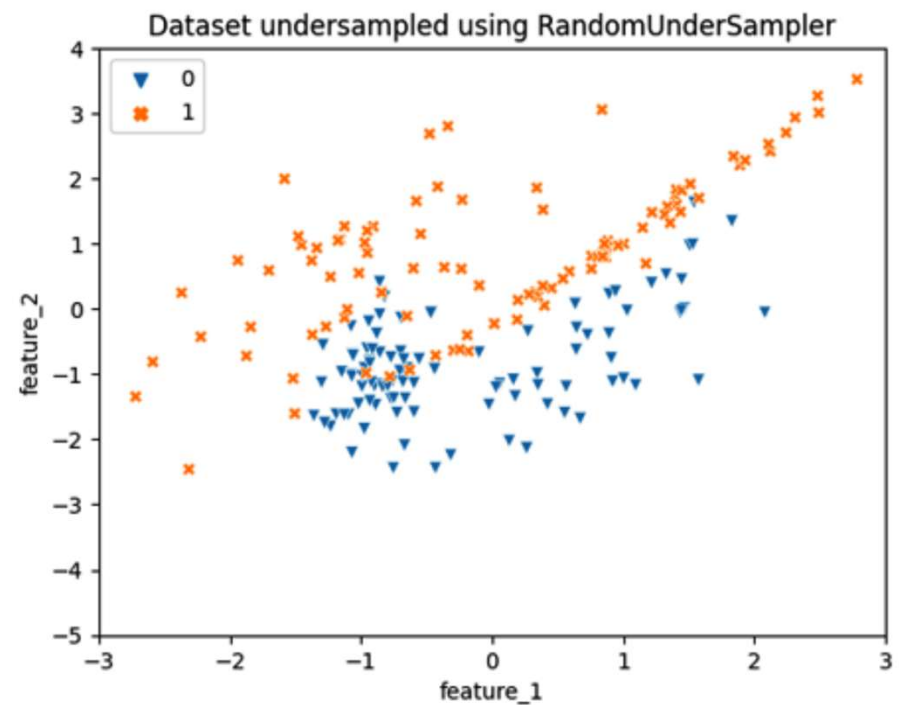
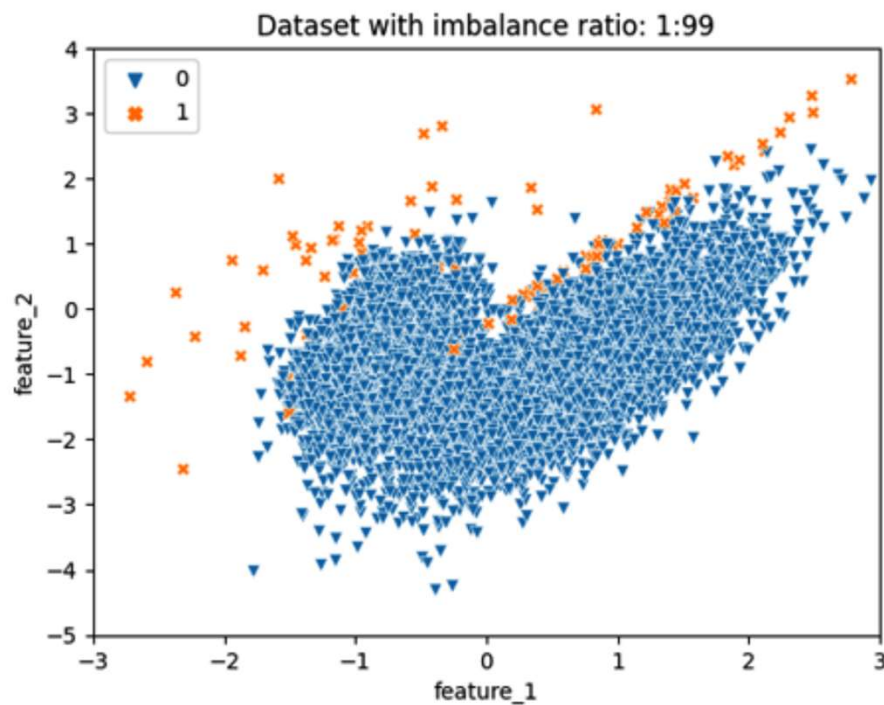
Removing examples uniformly

- Random UnderSampling (RUS)
 - Randomly remove majority class samples until balance is achieved.
- ClusterCentroids
 - Replace groups of majority samples with cluster centroids.

RUS

- **Step-by-Step Process**
 - Count class distribution
 - Randomly select majority samples
 - Remove remaining majority samples
 - Create balanced dataset

1. Random UnderSampling (RUS)



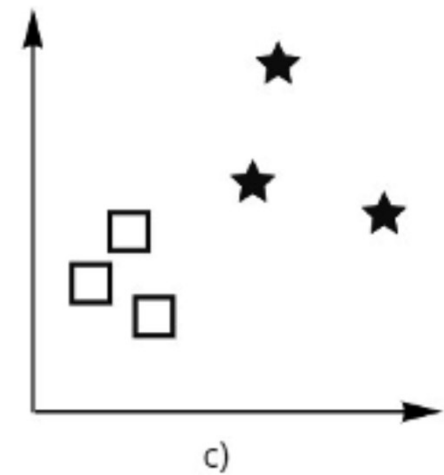
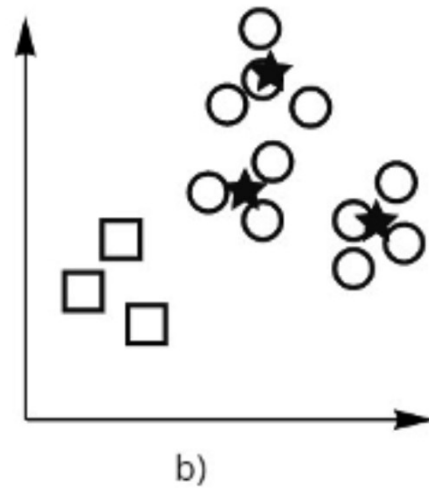
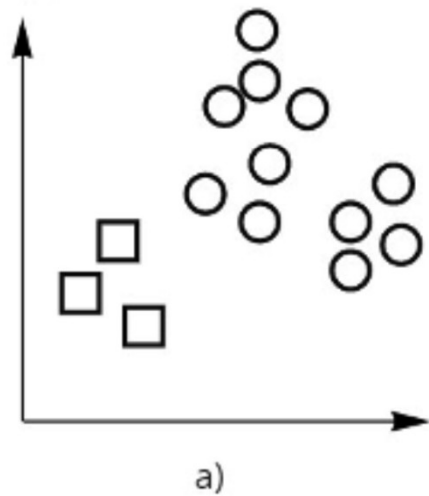
- Plotting datasets before and after undersampling using RandomUnderSampler

2. ClusterCentroids

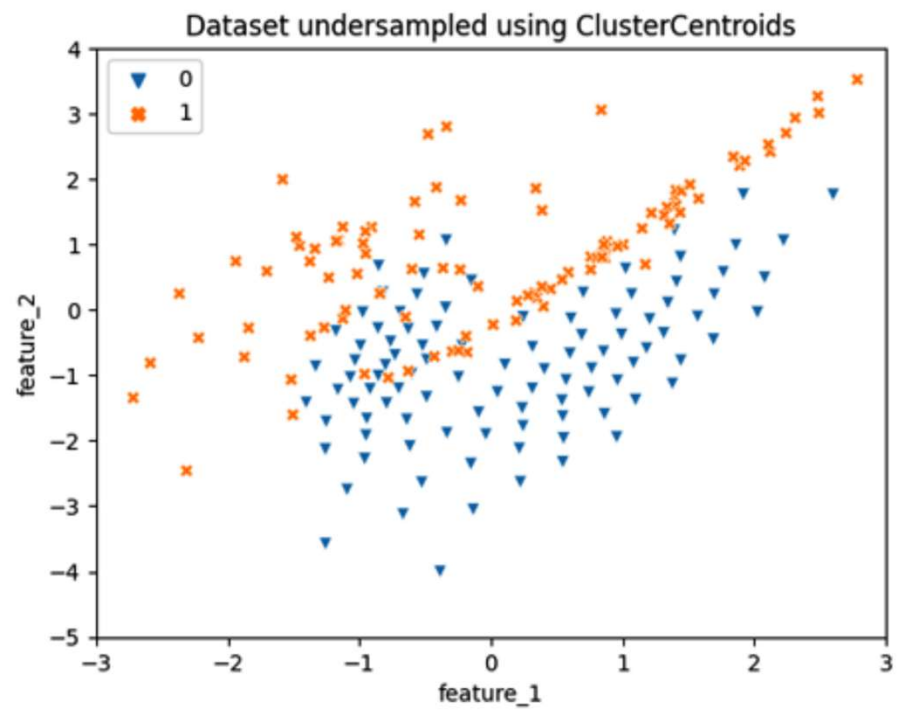
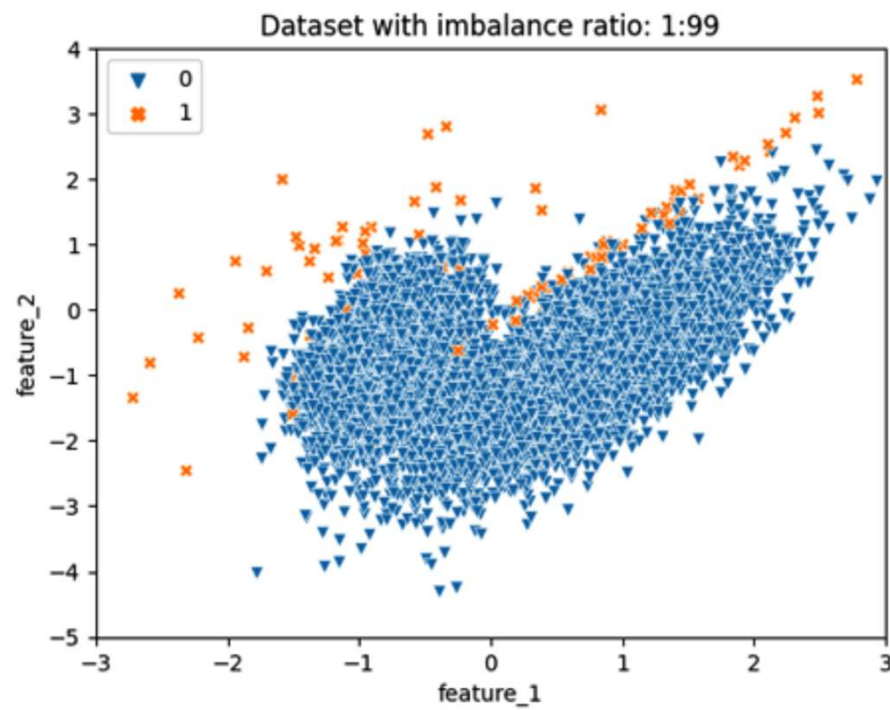
- Replace groups of majority samples with cluster centroids using k-means clustering.
- **Step-by-Step**
 - Apply K-means on majority class
 - Create N clusters
 - Compute centroids
 - Replace original samples with centroids

Illustrating how the ClusterCentroids method works

- Majority Class Sample
- Minority Class Sample
- ★—Cluster Centroid Point



Plotting datasets before and after undersampling using ClusterCentroids



Strategies for removing noisy observations

- **ENN**: Edited Nearest Neighbors
- **RENN**:
- **ALLKNN**:

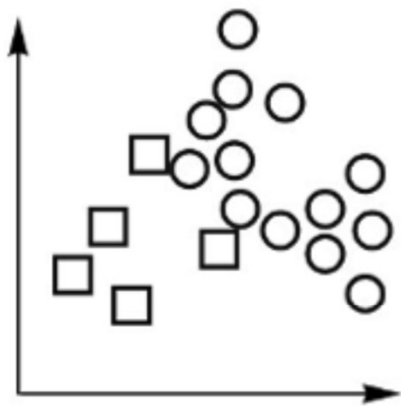
Edited Nearest Neighbors (ENN)

- ENN is an undersampling technique used mainly for imbalanced datasets. Instead of randomly removing majority samples, it *edits* the dataset by removing “noisy” or “misleading” points.
- Core Idea
 - A sample is removed if it is misclassified by its nearest neighbors.
 - So, ENN tries to clean the dataset by deleting points that:
 - are likely noise
 - lie near class boundaries
 - are surrounded mostly by opposite-class samples.

Step-by-step Working (ENN)

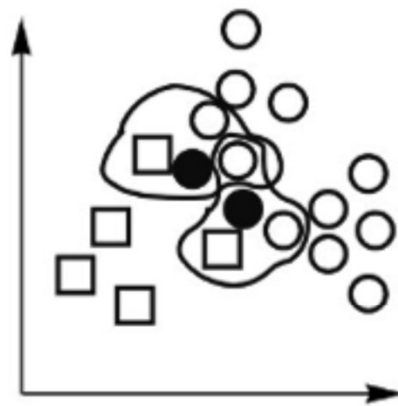
- Assume you have a dataset with majority and minority classes:
- **1. Choose a value of k**
 - Usually: $k = 3$ (common default)
- **2. For each data point:**
 - Take a sample x_i
 - Find its k nearest neighbors
 - Look at their class labels
- **3. Apply majority rule**
 - If most neighbors agree with the point's label → keep it
 - If most neighbors disagree with the point's label → remove it

Example: ENN



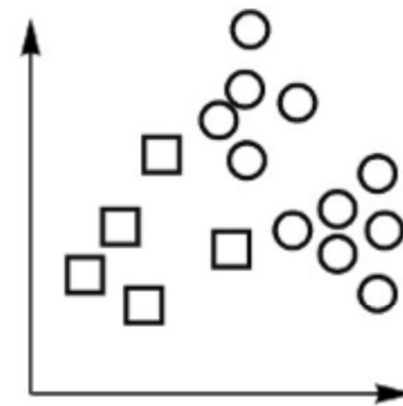
a)

○—Majority Class Sample



b)

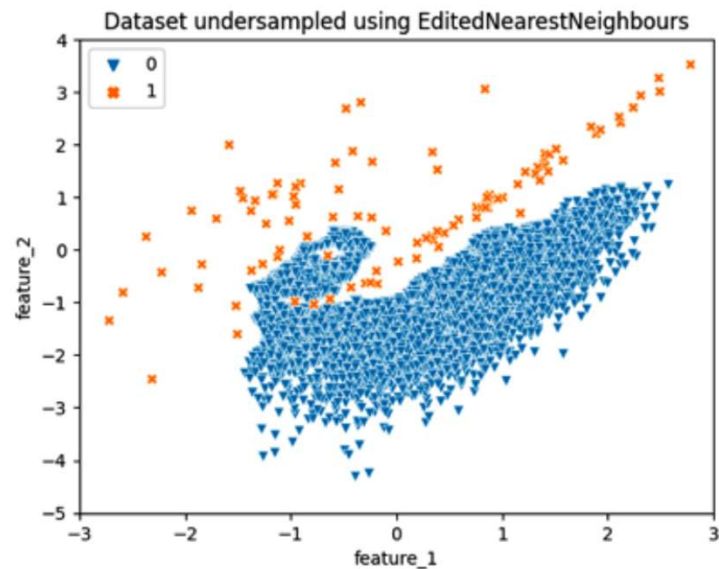
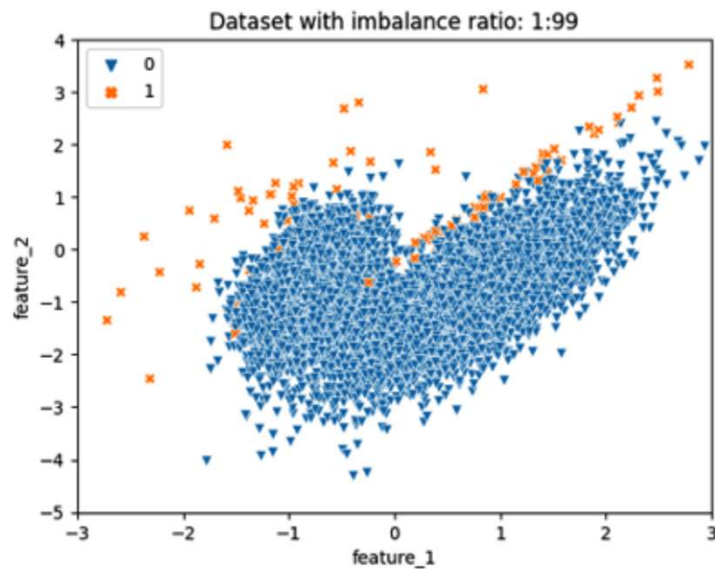
□—Minority Class Sample



c)

●—Majority Class Samples with
Minority Class Neighbors

- `from imblearn.under_sampling import EditedNearestNeighbours`
- `enn = EditedNearestNeighbours(sampling_strategy='auto', n_neighbors=200, kind_sel='all')`
- `X_res, y_res = enn.fit_resample(X, y)`
- `print('Resampled dataset shape %s' % Counter(y_res))`
- =====
- Resampled dataset shape Counter({0: 7852, 1: 100})



RENN: Repeated Edited Nearest Neighbours

- RENN is an improved version of ENN (Edited Nearest Neighbors) used for cleaning noisy and overlapping data, especially in imbalanced datasets.
- We repeat the process followed in ENN until there are no more examples that can be removed or the maximum number of cycle counts has been reached.
- So instead of a single cleaning pass (like ENN), RENN keeps refining the dataset.

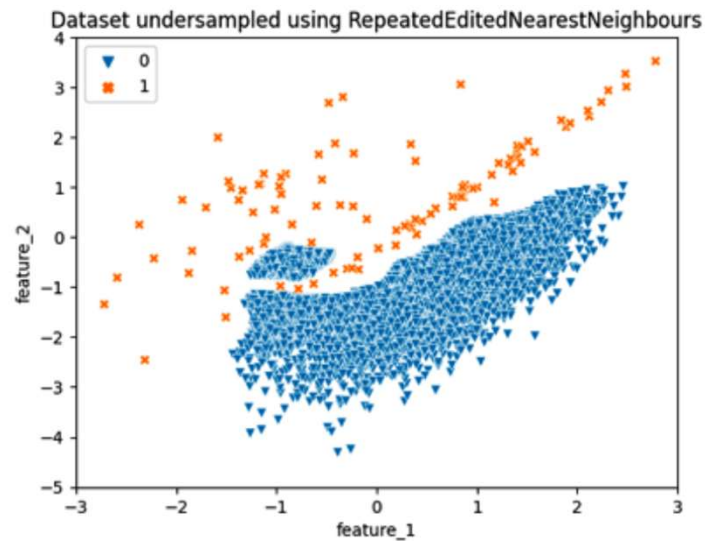
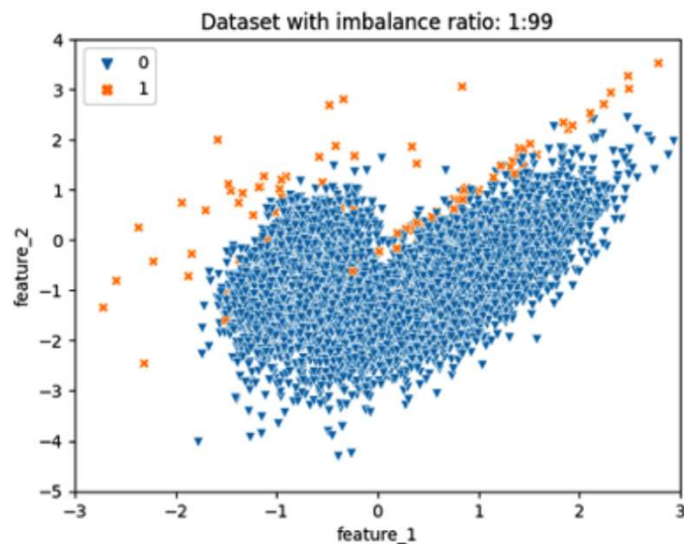
Step-by-step

- 1. Start with original dataset
 - You have majority + minority class samples.
- 2. Apply ENN once
 - For each sample, check its k nearest neighbors
 - Remove samples that are misclassified by neighbors
 - This gives a cleaned dataset (Round 1)
- 3. Repeat ENN again
 - Now on the reduced dataset:
 - Recalculate neighbors
 - Again remove misclassified samples
 - This becomes Round 2
- 4. Continue iterating
 - Repeat the process:
 - ENN → ENN → ENN →

RENN repeatedly applies ENN until the dataset becomes locally consistent and no further noisy samples remain.

Example: RENN

- This algorithm also removes the noisy data. It is stronger in removing the boundary examples as the algorithm is repeated several times



AllKNN: All k-Nearest Neighbors

- AllKNN is an advanced version of ENN (Edited Nearest Neighbors) used for cleaning overlapping and noisy data in imbalanced datasets.
- It is designed to be more adaptive than ENN and RENN.
- We repeat ENN but with the number of neighbors going from 1 to K.
- Instead of using a single fixed k, AllKNN:
 - Applies ENN multiple times while gradually increasing the number of neighbors (k).

Step-by-step Working

- **1. Start with dataset** (original imbalanced data)
- **2. Set initial k:** Start with a small value, usually:
 - $k = 1$ or $k = 3$
- **3. Apply ENN using current k**
 - For each sample:
 - Find k nearest neighbors
 - If misclassified by majority → remove it
- **4. Increase k gradually**
 - After one full ENN pass:
 - Increase k (e.g., $1 \rightarrow 3 \rightarrow 5 \rightarrow \dots$)
 - Apply ENN again on updated dataset
- **5. Repeat process**
 - Continue:
 - $\text{ENN}(k=1) \rightarrow \text{ENN}(k=3) \rightarrow \text{ENN}(k=5) \rightarrow \dots$
 - Until:
 - Maximum k is reached (usually predefined)
 - Or no further significant removals occur

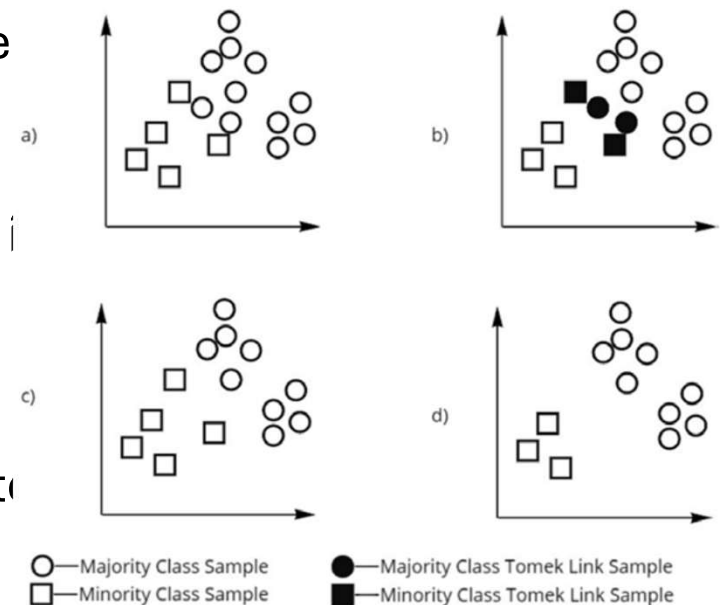
AllKNN repeatedly applies ENN while increasing k, enabling multi-level adaptive noise removal for cleaner decision boundaries.

ENN vs RENN vs AllKNN (Comparison)

Method	Strategy	Strength	Behavior
ENN	Single pass, fixed k	Medium	Simple cleaning
RENN	Repeat ENN until stable	Strong	Iterative refinement
AllKNN	Increase k gradually	Very strong	Multi-level adaptive cleaning

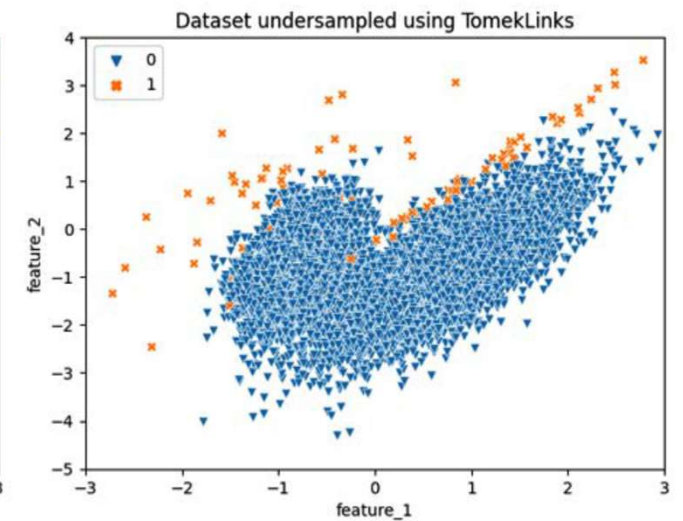
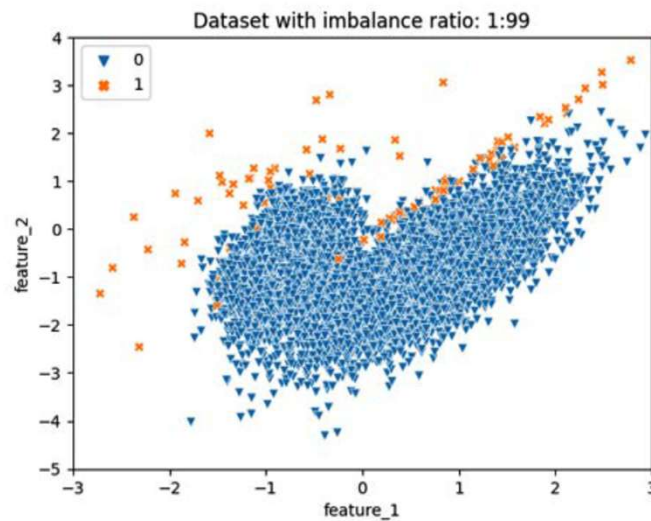
Tomek links

- In 1976, Ivan Tomek proposed the idea of Tomek links.
- Two examples are said to form Tomek links if they belong to two different classes, and there is no third point with a shorter distance to them than the distance between the two points.
- These points are part of the noise, and we can eliminate the majority member or both points to reduce noise.



if two points are from different classes, they should not be nearest to each other

- `from imblearn.under_sampling import TomekLinks`
- `tklinks = TomekLinks(sampling_strategy='auto')`
- `X_res, y_res = tklinks.fit_resample(X, y)`
- `print('Resampled dataset shape %s' % Counter(y_res))`
- =====
- Resampled dataset shape Counter({0: 9875, 1: 100})



Thanks